Okay. I'd like to adopt your plain JAX + minimal equinox plan and architectural recommendations. I have a minimal base for a VAMP implementation in GitHub under dmacd/vamp. I'd like to construct a plan for codex to follow that will

- Build out and validate a plain JAX TinyStories model as you outlined above
- builds and trains a tinyshakespeare-scale version of the TinyStories / GPT Neo architecture with the same jax structure and lora interfaces, suitable for smoke testing
- Integrate cleanly with the VAMP graph data structures and

Show more

Worked for 18m 6s

Recommended implementation plan

I inspected `dmacd/vamp`. The central conclusion is:

> **The existing immutable graph topology is worth preserving, but its current dense-parameter composition, MNIST-specific backend, and canvas-specific task interfaces should not be extended directly to language models.**

The present graph already uses frozen nodes and returns a new graph when a child is added. It reconstructs an effective parameter state by summing deltas along the parent path. That is a sound starting point.

However, `ModelBackend` currently requires canvases, labels, reconstruction, and a model-specific observed-energy method, while `TaskDataset` specifically stores MNIST images and materializes 32×32 canvases. Extending these with branches for token sequences would create exactly the kind of awkward cross-domain interface that should be avoided.

The project's own style rules already require persistent functional data, discourage mutable state and `Any`, and direct agents to update `PLAN.md` and `DESIGN.md` as implementation proceeds.

1. Architectural decisions to freeze before implementation

1.1 Use plain JAX model values, not Equinox modules

For the initial implementation, "minimal Equinox" should mean **zero Equinox dependency in the canonical model**.

Use:

- Frozen configuration dataclasses
- `NamedTuple` parameter records, which are already JAX PyTrees
- Tuples for layer stacks
- Pure `init_*`, `apply_*`, `loss_*`, and `train_step` functions
- Explicit Optax state
- Explicit random keys
- Explicit generation cache values when a cache is eventually added

Do not use:

- `eqx.Module` as the model representation
- Equinox serialization as the checkpoint format
- Stateful model wrappers
- A mutable "session" or "trainer" object
- NNX anywhere in this implementation

Equinox can later be admitted for an isolated utility such as filtered transformations only when it removes substantial boilerplate. No Equinox type should appear in the graph, checkpoint schema, LoRA representation, or public VAMP interfaces.

This is consistent with the existing VAE implementation, which already represents parameters as nested dictionaries and tuples and applies them through pure JAX functions.

1.2 Keep the frozen base model outside the VAMP graph

The TinyStories base parameters are immutable shared infrastructure:

```
Frozen GPT-Neo base
        +
Selected VAMP path of LoRA edges
        =
Effective language model
```

Do not copy the base model into graph nodes, and do not include it in every node's memory accounting.

Persistent memory should be reported as:

$$M_{\text{persistent}} = M_{\text{base, once}} + \sum_{e} M_{\text{LoRA edge},e} + M_{\text{address keys}} + M_{\text{graph metadata}}.$$

Runtime caches, packed candidate banks, optimizer state, and compilation artifacts should be reported separately.

1.3 Each graph edge is an independent LoRA update

This is the most important semantic choice.

Suppose an edge e has LoRA factors

$$A_e \in \mathbb{R}^{d_{\text{in}} \times r}, \qquad B_e \in \mathbb{R}^{r \times d_{\text{out}}}.$$

For a node n, let $P(n)$ denote the edges on its root-to-node path. At a linear projection, the effective operation should be

$$y = xW + b + \sum_{e \in P(n)} s_e (x A_e) B_e.$$

In particular:

- **Do not sum the A factors and B factors separately.**
- **Do not store coordinate differences between child and parent A/B factors and then treat those differences as LoRA modules.**
- Sum the actual low-rank **output contributions** of the edges on the path.

This gives the intended VAMP interpretation:

- Every new task adds one immutable residual edge.
- Parent-path competence is inherited.
- The new edge learns only what the parent path lacks.
- Old paths remain exactly unchanged.
- Effective rank grows with path depth.
- Later compression can approximate the sum of low-rank updates with a smaller factorization.

This also makes "stored forgetting" structurally distinguishable from routing failure.

1.4 Generalize graph topology, not model semantics

Refactor `DenseMemoryGraph` into a generic immutable topology whose incoming edge payload is parameterized:

```python
NodeId = NewType("NodeId", str)
TaskId = NewType("TaskId", str)

EdgePayloadT = TypeVar("EdgePayloadT")


@dataclass(frozen=True)
class MemoryNode(Generic[EdgePayloadT]):
    node_id: NodeId
    parent_id: NodeId | None
    trained_task: TaskId | None
    train_stage: int
    depth: int
    incoming_edge: EdgePayloadT | None


@dataclass(frozen=True)
class MemoryGraph(Generic[EdgePayloadT]):
    nodes: tuple[MemoryNode[EdgePayloadT], ...]
```

Then define two model-specific interpretations:

```
Dense MNIST graph:
    incoming edge = dense PyTree delta
    effective params = root params + pathwise dense deltas

Language VAMP graph:
    incoming edge = LoRA edge PyTree
    effective model = frozen base + pathwise LoRA contributions
```

The reusable graph layer should provide only:

- Node insertion
- Node lookup
- Parent and child traversal
- Root-to-node path
- Node ordering
- Depth
- Path-incidence matrices
- Payload byte accounting through an injected or model-specific function
- Generic visualization metadata

It should not know how a dense delta or LoRA adapter changes a model.

1.5 Compile the immutable graph into fixed-shape JAX arrays

The host-side graph remains an immutable Python value. Before training or routing, compile it into a fixed-shape JAX representation:

```python
class PackedLoraMemory(NamedTuple):
    edge_bank: LoraEdgeBank
    node_path_matrix: jax.Array   # [max_nodes, max_edges]
    valid_node_mask: jax.Array    # [max_nodes]
    valid_edge_mask: jax.Array    # [max_edges]
```

`node_path_matrix[n, e]` is one when edge e belongs to node n's path.

The maximum node and edge counts should be fixed by the experiment curriculum. Unused entries are padded and masked. This avoids compiling a new executable every time the graph grows.

That is especially important because the repository already identifies variable candidate and batch shapes as a source of repeated XLA compilation and GPU stalls in the current FabricPC benchmark.

1.6 Separate routing input from competence evaluation

For every held-out document, split tokens into:

```
[address prefix][evaluation suffix]
```

The address is selected from the prefix only. Competence is measured on the suffix only.

For example:

$$x = (x_{1:P}, x_{P+1:P+S}).$$

Routing uses

$$E_n(x_{1:P}) = -\frac{1}{P-1}\sum_{t=1}^{P-1} \log p_n(x_{t+1} \mid x_{\le t}),$$

while reported continuation competence uses

$$L_n(x_{P+1:P+S} \mid x_{1:P}).$$

This prevents optimistic double-dipping in which the same tokens both select the model and measure its performance.

The router must never receive:

- Task ID
- Task boundary metadata
- Corpus partition ID
- Oracle node ID
- Labels derived from those values

The evaluator retains them separately for scoring routing accuracy.

---

2. Plain-JAX GPT-Neo implementation

2.1 Static configuration

Define a frozen static configuration:

```python
@dataclass(frozen=True)
class GptNeoConfig:
    vocab_size: int
    max_position_embeddings: int
    hidden_size: int
    intermediate_size: int
    num_layers: int
    num_heads: int
    attention_types: tuple[Literal["global", "local"], ...]
    local_window_size: int
    layer_norm_epsilon: float
    activation: Literal["gelu_new"]
    embedding_dropout: float
    attention_dropout: float
    residual_dropout: float
```

The released TinyStories-8M checkpoint uses GPT-Neo with width 256, eight layers, 16 heads, a 2,048-token position limit, alternating global and local attention, local window 256, `gelu_new`, and vocabulary size 50,257. Hugging Face

2.2 Typed parameter tree

Use immutable tuple records:

```python
class LinearParams(NamedTuple):
    kernel: jax.Array
    bias: jax.Array | None


class LayerNormParams(NamedTuple):
    scale: jax.Array
    bias: jax.Array


class AttentionParams(NamedTuple):
    query: LinearParams
    key: LinearParams
    value: LinearParams
    output: LinearParams


class MlpParams(NamedTuple):
    input_projection: LinearParams
    output_projection: LinearParams


class TransformerBlockParams(NamedTuple):
    attention_norm: LayerNormParams
    attention: AttentionParams
    mlp_norm: LayerNormParams
    mlp: MlpParams


class GptNeoParams(NamedTuple):
    token_embedding: jax.Array
    position_embedding: jax.Array
    blocks: tuple[TransformerBlockParams, ...]
    final_norm: LayerNormParams
```

Do not store a separate language-model head. Compute:

$$\text{logits} = hE^\top$$

using the token embedding matrix E, matching the tied embedding/output behavior.

The parity implementation must also reproduce the relevant HF GPT-Neo details: bias-free query/key/value projections, biased output and MLP projections, pre-layer-normalized blocks, float32 attention-score computation, and tied input/output embeddings. GitHub

2.3 Forward interfaces

Split the model into composable pure functions:

```python
def embed_tokens(
    params: GptNeoParams,
    token_ids: jax.Array,
    position_ids: jax.Array,
) -> jax.Array:
    ...


def apply_gpt_neo_embeddings(
    params: GptNeoParams,
    lora_memory: PackedLoraMemory | None,
    edge_coefficients: jax.Array | None,
    input_embeddings: jax.Array,
    attention_mask: jax.Array,
    *,
    capture: CaptureSpec,
) -> ForwardResult:
    ...


def apply_gpt_neo(
```

```
    params: GptNeoParams,
    lora_memory: PackedLoraMemory | None,
    edge_coefficients: jax.Array | None,
    token_ids: jax.Array,
    attention_mask: jax.Array,
    *,
    capture: CaptureSpec,
) -> ForwardResult:
    ...
```

`ForwardResult` should have a fixed PyTree shape:

```
class ForwardResult(NamedTuple):
    logits: jax.Array
    final_hidden: jax.Array
```

The embeddings-level function leaves a clean path for later soft-prompt optimization and prompt-to-LoRA baking without changing transformer internals.

Do not implement a mutable hook or activation registry. Intermediate capture is controlled by a static `CaptureSpec` and returned explicitly.

2.4 Generation

Implement generation in two stages:

1. **Parity generation:** recompute the full prefix for every generated token.
2. **Cached generation:** add a fixed-shape immutable KV-cache only after all checkpoint and LoRA parity tests pass.

The first implementation is inefficient but small, inspectable, and much easier to validate. TinyStories-8M is small enough that this is acceptable for initial interactive work.

---

3. LoRA representation and graph execution

3.1 Supported sites

The first complete LoRA interface should support every transformer linear projection:

```
attention.query
attention.key
attention.value
attention.output
mlp.input_projection
mlp.output_projection
```

Embedding and layer-normalization adaptation are explicitly outside the first milestone.

Use a global fixed rank for the initial graph. The target-site set is also static for a run. This satisfies the present uniform-shape restriction without building an arbitrary rewrite language.

A projection adapter has:

```
class LoraProjection(NamedTuple):
    left: jax.Array    # [input_dim, rank]
    right: jax.Array   # [rank, output_dim]


class LoraBlock(NamedTuple):
    query: LoraProjection
    key: LoraProjection
    value: LoraProjection
```

```
    attention_output: LoraProjection
    mlp_input: LoraProjection
    mlp_output: LoraProjection


class LoraEdge(NamedTuple):
    blocks: tuple[LoraBlock, ...]
```

Disabled target sites can either be omitted through a static tree structure or represented with a static target mask. The structure must be identical for every edge in a given run.

Initialize a new edge with a random left factor and zero right factor, producing an exactly zero functional update while preserving usable gradients.

3.2 Efficient path application

Stack corresponding factors across edges:

```
A: [max_edges, input_dim, rank]
B: [max_edges, rank, output_dim]
c: [max_edges]
```

At one linear site:

$$u_{\ldots er} = \sum_i x_{\ldots i} A_{eir}$$

$$v_{\ldots eo} = \sum_r u_{\ldots er} B_{ero}$$

$$\Delta y_{\ldots o} = \sum_e c_e v_{\ldots eo}.$$

For a hard node selection, c is its binary path mask.

For continuous addressing, c is a continuous edge-coefficient vector.

This one operation supports:

- Training a new edge over a frozen parent path
- Evaluating a selected node
- Differentiable mixtures over nodes
- Branching graph paths
- Eventual adapter-path compression

3.3 Training a new task edge

Given selected parent node p:

1. Freeze the base model.
2. Freeze every committed edge.
3. Obtain the binary path coefficients c_p.
4. Initialize a new zero-effect LoRA edge in the next padded edge slot.
5. Train only that edge while applying c_p plus the new edge.
6. Commit the trained edge as a child of p.
7. Derive and store the new node's content-address key.
8. Return a new immutable memory graph and packed memory value.

The gradient function should accept only the new edge as its differentiated argument:

```


def task_loss(
    candidate_edge: LoraEdge,
    frozen_base: GptNeoParams,
    frozen_edge_bank: LoraEdgeBank,
    parent_path: jax.Array,
    batch: TokenBatch,
```

```
) -> jax.Array:
    ...
```

This makes accidental updates to the base or previous memories difficult by construction.

---

4. The three addressing modes

All three routers operate over the same node ordering and the same packed path-incidence matrix.

4.1 Exhaustive prefix-energy search

For every valid graph node:

1. Apply its hard path mask.
2. calculate normalized teacher-forced prefix NLL;
3. select the lowest-energy node.

Return more than just the winner:

```
class AddressResult(NamedTuple):
    selected_indices: jax.Array
    node_probabilities: jax.Array
    node_scores: jax.Array
    score_margin: jax.Array
    entropy: jax.Array
```

The host-level result also records selected node IDs and cost measurements.

This is the gold-standard router for early experiments. It is not intended to scale well.

For parent selection during known-boundary training, evaluate mean prefix NLL over a deterministic training probe and select the lowest-energy parent. The task boundary determines when to create a node, but the task ID does not directly choose the parent.

4.2 Hopfield content lookup

Define a replaceable pure content encoder:

```
ContentEncoder = Callable[
    [GptNeoParams, TokenBatch],
    jax.Array,  # [batch, embedding_dim]
]
```

The initial default should be:

1. Run the **frozen base model without VAMP adapters**.
2. Mean-pool final hidden states over valid address-prefix tokens.
3. L2-normalize each result.
4. Build a node key as the normalized centroid of deterministic training-prefix embeddings.

For node keys K_n and query q:

$$a_n = \mathrm{softmax}\left(\beta K_n^\top q\right).$$

Pure Hopfield mode selects $\arg\max_n a_n$. Also report top-k recall and entropy.

The key-builder and content-encoder interfaces must remain swappable because later candidates may include:

- Last-token states
- Layerwise pooled states
- Gradient embeddings

- Residual-response signatures
- Multiple prototypes per node
- Adapter-conditioned keys

Do not block the first benchmark on determining the ideal embedding.

4.3 EBT-inspired differentiable address refinement

Let $z \in \mathbb{R}^N$ be per-sequence address logits and

$$w = \mathrm{softmax}(z/\tau).$$

Convert node weights into edge coefficients using the path matrix P:

$$c = w^\top P.$$

The model then applies a continuous weighted sum of edge LoRA contributions at every target projection.

Optimize only z:

$$z^\star = \arg\min_z \left[L_{\text{prefix}}\big(c(z)\big) + \lambda_H H(w) \right].$$

Here $+\lambda_H H(w)$ penalizes diffuse high-entropy addresses when $\lambda_H > 0$.

The implementation should support:

- Uniform initialization
- Hopfield-logit initialization
- Full-node refinement
- Hopfield top-k masked refinement
- Fixed optimization step count
- Fixed temperature or a static annealing schedule

After refinement, report both:

1. The soft-mixture prefix and suffix NLL
2. The hard node selected by $\arg\max w$, with its discrete suffix NLL

The hard result is the primary task-free routing result. The soft result helps determine whether the graph contains useful compositional mixtures that no single node captures.

This method performs one base-model forward per refinement step plus the stacked LoRA-edge work. It does **not** need to run the complete base transformer separately for every candidate.

---

5. TinyShakespeare model and smoke-test path

5.1 Standard smoke configuration

Use the same GPT-Neo implementation with:

```
Tokenizer: character-level
Vocabulary: derived deterministically from training text
Context length: 256
Hidden width: 128
Layers: 4
Heads: 4
MLP width: 512
Attention: alternating global/local
Local window: 64
Dropout: initially zero
```

This is approximately an 0.8-million-parameter model, depending on vocabulary size.

Also define a unit-test configuration:

```
Context length: 64
Hidden width: 64
Layers: 2
Heads: 4
MLP width: 256
```

The TinyShakespeare corpus is only around a megabyte, making it appropriate for fast local training and deliberate overfitting tests. GitHub

5.2 TinyShakespeare acceptance criteria

The full TinyShakespeare phase is complete when:

- A fixed training batch can be deliberately overfit.
- Validation NLL decreases during an ordinary training run.
- Checkpoint save/load preserves logits.
- Fixed-seed greedy generation is repeatable.
- A zero-effect LoRA path exactly reproduces base logits.
- A task LoRA edge improves NLL on its task.
- The frozen base checksum is unchanged after edge training.
- Previously committed node outputs remain unchanged after adding later nodes.
- The same LoRA and routing functions work for both TinyShakespeare and TinyStories configurations.

Do not treat a particular public nanoGPT validation loss as a hard gate because the architecture, tokenizer, optimizer, context distribution, and training budget differ.

---

6. TinyStories-8M conversion and parity validation

The model repository supplies a roughly 112 MB PyTorch checkpoint plus GPT-2 tokenizer files, rather than a native JAX checkpoint. Hugging Face +1 The tokenizer uses GPT-2 vocabulary machinery, a 2,048-token maximum, and `<|endoftext|>` as BOS/EOS, with no ordinary pad token. Hugging Face

Create explicit extras:

```
lm = [
    "huggingface_hub",
    "tokenizers",
    "safetensors",
]

hf-convert = [
    "torch",
    "transformers",
]
```

PyTorch and Transformers are used only by:

```
scripts/convert_tinystories_8m.py
tests/integration/test_tinystories_hf_parity.py
```

They should not be imported by normal training, generation, routing, or benchmark modules.

Conversion output

Write:

```
checkpoints/tinystories_8m_jax/
    model.safetensors
    manifest.json
    tokenizer.json
    tokenizer_config.json
```

```
    vocab.json
    merges.txt
```

The manifest records:

- Source repository and immutable revision
- Source checkpoint hash
- Converter version
- Model configuration
- Canonical parameter names
- Array shapes and dtypes
- Tokenizer hashes
- Date and environment metadata

The converter must fail on missing or unexpected source keys.

Parity ladder

Validate in this order:

1. Tokenization parity
2. Embedding output
3. Position handling
4. Global causal attention mask
5. Local-window mask boundaries
6. Each block’s post-attention residual
7. Each block’s post-MLP residual
8. Final hidden state
9. Full logits
10. Mean token NLL
11. Greedy generated token sequence

Use fixed integer token arrays first, not natural-language prompts.

A reasonable initial float32 criterion is:

```
np.testing.assert_allclose(
    jax_output,
    torch_output,
    rtol=2e-4,
    atol=2e-4,
)
```

Do not relax the tolerance globally merely to get a passing test. Record the maximum and mean error per layer and identify the operation producing the discrepancy.

Default `pytest` must not download the model. Tiny random-model tests run in ordinary CI; checkpoint parity is an explicitly marked integration suite.

---

7. Language-task and continual-learning interfaces

Do not adapt the current MNIST `TaskDataset` into a union of images and token arrays.

Introduce language-specific immutable records:

```
@dataclass(frozen=True)
class LanguageTask:
    task_id: TaskId
    display_name: str
    train_data: TokenDatasetRef
    validation_data: TokenDatasetRef
```

```
    address_probe: TokenDatasetRef
    metadata: PMap[str, JsonValue]


class TokenBatch(NamedTuple):
    input_ids: jax.Array
    target_ids: jax.Array
    attention_mask: jax.Array
    loss_mask: jax.Array


@dataclass(frozen=True)
class LanguageCurriculum:
    curriculum_id: str
    tasks: tuple[LanguageTask, ...]
    max_nodes: int
    max_edges: int
    address_prefix_lengths: tuple[int, ...]
    evaluation_suffix_length: int
```

The task ID is available to the training orchestrator and evaluator, but not to any address function.

Functional run state

```
@dataclass(frozen=True)
class LanguageVampRun:
    base_checkpoint: BaseCheckpointRef
    graph: MemoryGraph[LoraEdge]
    address_book: AddressBook
    rng_key: jax.Array
    completed_tasks: tuple[TaskId, ...]
    metrics: tuple[MetricRecord, ...]
```

Each stage is a pure transition:

$$(\text{run}, \text{task}) \mapsto (\text{run}', \text{stage result}).$$

Host-side loops are fine. Hidden mutation of model state is not.

Do not force reuse of `ModelBackend`

The existing backend abstraction is appropriate for the current generative-canvas experiments, but not for language modeling. It includes `reconstruct`, canvas arrays, labels, and a single model-specific observed-energy interpretation.

Create a language-specific set of pure operations first. Once the LM and MNIST implementations reveal a genuinely common lifecycle, extract only that lifecycle:

```
choose parent
train incoming edge
commit immutable node
derive address representation
evaluate stored and routed competence
record memory and cost
```

Do not manufacture a large generic backend protocol before both implementations exist.

---

8. Initial curricula

Each dataset should have three kinds of curriculum:

1. A mechanistic curriculum with a strong known signal
2. A natural content curriculum
3. A negative-control curriculum

8.1 TinyShakespeare curricula

A. Character-permutation curriculum

Create four fixed seeded permutations over alphabetic characters while preserving:

- Whitespace
- Newlines
- Punctuation
- Digits
- Special tokens

Apply each permutation to the same underlying document distribution, with document-level train/validation/test separation.

Purpose:

- Strongly tests whether a new LoRA edge can store a distinct transformation.
- Gives routing an identifiable signal.
- Mirrors the role of PermutedMNIST.
- Exposes path, packing, masking, and task-leakage bugs quickly.

B. Corpus-region curriculum

Split the corpus into four disjoint document or contiguous macro-regions before creating token windows.

Purpose:

- Tests natural vocabulary, character, speaker, and style differences.
- Provides a less artificial routing problem.
- Avoids requiring a fragile semantic play parser in the first implementation.

No training or evaluation window may cross a region or split boundary.

C. Stable-hash negative control

Assign documents to tasks by a stable content hash independent of their linguistic content.

Expected behavior:

- Stored task adapters may have only small differences.
- Task-node routing accuracy should remain near chance.
- A high routing score indicates leakage or an invalid evaluation protocol.

8.2 TinyStories curricula

The public dataset contains roughly 2.1 million training stories and approximately 22,000 validation stories, and the repository also provides V2/GPT-4 and metadata-oriented artifacts.

Hugging Face +2

A. Token-permutation curriculum

Select a fixed set of common, non-special token IDs and create four seeded permutations. Preserve EOS and designated punctuation or structural tokens.

Purpose:

- Fast, high-signal test of TinyStories LoRA storage.
- Same VAMP and router code as the real benchmark.
- More demanding than TinyShakespeare while remaining diagnosable.

B. Topic curriculum

Construct balanced, deterministic topic buckets, initially using explicit lexical rules with overlap rejection. A practical first set is:

```
animals
vehicles/tools
family/friendship/home
fantasy/royalty
```

Require:

- A minimum evidence score
- A unique winning category
- A margin over the second category
- Equalized task sizes
- Story-level splits before token packing
- Rules and lexicons stored in the run manifest

Later, replace or augment this with provided metadata or frozen-model clustering.

Use the newer V2/GPT-4 material where practical and hash-deduplicate exact stories against evaluation sources. Where complete overlap verification is not available, describe the experiment as in-domain continual adaptation rather than unseen-distribution generalization.

C. Stable-hash negative control

Assign stories to tasks solely from normalized-text hashes.

As with TinyShakespeare, routing should remain near chance because task membership contains no content signal.

Evaluation prefix sweep

Use a standard prefix sweep:

```
TinyShakespeare: 32, 64, 128 characters
TinyStories:     16, 32, 64, 128 tokens
```

Use a separate fixed suffix length for competence scoring.

---

9. Baseline matrix

Every curriculum should report at least these methods:

Method	Purpose
Frozen base	No-adaptation floor
Sequential single LoRA	Ordinary adapter forgetting baseline
Independent root LoRA per task	Per-task adapter ceiling without transfer
VAMP task-oracle path	Stored competence
VAMP exhaustive router	Gold-standard task-free routing
VAMP Hopfield router	Cheap content routing
VAMP EBT router, uniform start	Continuous addressing without content prior
VAMP EBT router, Hopfield start	Combined retrieval and refinement
Random valid node	Routing negative control

For TinyShakespeare, an optional sequential full-model fine-tuning baseline is affordable. It is not necessary for TinyStories-8M during the initial milestone.

The independent-root baseline and inherited-parent VAMP baseline are essential for measuring whether graph inheritance produces transfer rather than merely adding storage.

---

10. Metric decomposition

Let n_j be the node committed for task j, $\hat{n}_r(x)$ the node chosen by router r, and $L_s(n, x)$ the evaluation-suffix NLL under node n.

10.1 Stored competence

Report:

- Frozen-base suffix NLL and perplexity
- Task-node suffix NLL $L_s(n_j, x)$
- Best available node suffix NLL
- Independent-task LoRA suffix NLL
- Improvement over frozen base
- Improvement or deficit relative to independent LoRA
- Old-node logit and NLL drift after every later graph commit
- Checksum stability of base and committed edges

Stored forgetting for task j at stage t:

$$F_{t,j}^{\text{stored}} = L_{t,j}^{\text{oracle}} - \min_{s \in [j,t]} L_{s,j}^{\text{oracle}}.$$

This should be numerically zero or within deterministic floating-point tolerance because old paths are immutable.

10.2 Routing competence

Report:

- Task-node top-1 accuracy
- Task-node top-k recall
- Agreement with lowest-prefix-NLL node
- Routed suffix NLL
- Task-oracle routing regret

$$R^{\text{task}}(x) = L_s(\hat{n}_r(x), x) - L_s(n_j, x)$$

- Best-node routing regret

$$R^{\text{best}}(x) = L_s(\hat{n}_r(x), x) - \min_n L_s(n, x)$$

- Address entropy
- Top-two margin
- Confusion matrix
- Metrics by address-prefix length
- Metrics as additional graph nodes are introduced

Routing forgetting should be measured separately from stored forgetting:

$$F_{t,j}^{\text{route}} = L_{t,j}^{\text{routed}} - \min_{s \in [j,t]} L_{s,j}^{\text{routed}}.$$

This captures candidate interference even when task parameters remain perfectly stored.

10.3 Transfer

Report:

- New-task NLL under root
- New-task NLL under selected parent before training
- Parent advantage over root
- First-step and fixed-budget learning curves

- Updates and tokens required to reach a fixed NLL threshold
- Final VAMP child path versus independent root LoRA
- Selected-parent identity and parent-search energy
- Backward transfer, if any, on previously learned tasks

A direct transfer measure is:

$$T_j = L^{\text{initial}}_{\text{root}}(j) - L^{\text{initial}}_{\text{selected parent}}(j).$$

A negative value exposes harmful parent selection.

10.4 Memory

Report separately:

- Frozen base parameter count and bytes, once
- LoRA bytes per edge
- Total committed LoRA bytes
- Bytes along each effective path
- Address-key bytes per node
- Graph metadata bytes
- Packed runtime edge-bank bytes
- Optimizer-state peak bytes
- Bytes per task
- Bytes per unit of NLL improvement
- Effective uncompressed rank by path and projection

Do not count runtime padding as persistent graph storage.

10.5 Addressing cost

Report:

- Address-prefix tokens
- Candidates available
- Candidates actually scored
- Full model forward-equivalent token count
- Base-model forwards
- LoRA-edge evaluations
- Hopfield dot products
- EBT gradient steps
- EBT candidate or top-k mask size
- Cold compile time
- Warm steady-state latency
- Batch throughput
- Optional peak device allocation
- Selected-node model-evaluation cost after routing

JAX timings must call `block_until_ready()`. Cold compilation and warm execution must never be combined into one latency number.

---

11. Codex work packets

Each work packet should leave the repository green and should update `PLAN.md`. Durable decisions go into `DESIGN.md`, as required by the repository guidance.

Phase 0 — Record the architecture contract

Create:

```
docs/LM_VAMP_EXECUTION_PLAN.md
```

Update `DESIGN.md` with:

- Frozen base outside graph
- Generic graph topology
- Pathwise LoRA contribution semantics
- Fixed-shape packed memory
- Prefix/suffix evaluation separation
- Three router definitions
- No task IDs during routing
- PyTorch conversion boundary
- No mutable model representation

Update `PLAN.md` with the active phase and immediate gate.

Gate: Existing tests pass unchanged.

Phase 1 — Extract generic graph topology

Create or refactor toward:

```
src/apm/memory/graph.py
src/apm/memory/dense.py
```

Tasks:

- Move node topology and path traversal into generic `MemoryGraph`.
- Keep dense-MNIST payload composition in `dense.py`.
- Eliminate `ParamTree = Any`.
- Add path-incidence matrix construction.
- Update visualization to consume generic topology plus supplied stats.
- Migrate all existing Stage 1 callers rather than retaining compatibility aliases.

Tests:

- Linear path
- Branching paths
- Duplicate node rejection
- Unknown parent rejection
- Correct depth
- Correct path incidence
- Dense parameter reconstruction unchanged
- Existing MNIST tests and smoke benchmark remain green

The current graph tests already establish dense child reconstruction and addressed behavior and should continue to pass after migration.

Phase 2 — Implement pure-JAX GPT-Neo

Create:

```
src/apm/lm/config.py
src/apm/lm/parameters.py
src/apm/lm/gpt_neo.py
src/apm/lm/attention.py
src/apm/lm/losses.py
```

Tests:

- Parameter shapes
- Causal masking
- Local-window boundary behavior
- Padding masking
- No future-token influence
- `gelu_new`
- Weight tying
- JIT consistency
- Gradient existence

- Fixed capture structure
- Tiny random configuration CPU smoke

Gate: Tiny random model trains and overfits a fixed batch without any LoRA code.

Phase 3 — Implement pathwise LoRA edges

Create:

```
src/apm/lm/lora.py
src/apm/lm/lora_memory.py
```

Tests:

- Newly initialized edge has exactly zero output effect.
- One edge matches direct LoRA calculation.
- A two-edge path equals the sum of the two edge contributions.
- Sibling paths do not include each other’s edges.
- One-hot node coefficients reproduce hard path execution.
- Continuous node weights map correctly through the path matrix.
- Only the candidate edge receives gradients.
- Frozen base and committed-edge hashes remain unchanged.
- Padded invalid nodes and edges have no effect.

Gate: Two synthetic tasks can commit sibling or parent-child LoRA edges while preserving previous-node logits.

Phase 4 — TinyShakespeare base model and interactive workflow

Create:

```
src/apm/data/text/tinyshakespeare.py
src/apm/data/text/tokenization.py
src/apm/lm/training.py
src/apm/lm/checkpoint.py
src/apm/lm/generation.py
scripts/train_tinyshakespeare.py
notebooks/tinyshakespeare_vamp.ipynb
```

The notebook should import tested library functions rather than implementing algorithms in cells.

Gate:

- Base training run
- Checkpoint round trip
- Deterministic generation
- LoRA single-task overfit
- No mutable interactive session object

Phase 5 — Convert and validate TinyStories-8M

Create:

```
scripts/convert_tinystories_8m.py
tests/integration/test_tinystories_hf_parity.py
```

Gate:

- Manifest and checkpoint generated
- No missing/unexpected weights
- Layerwise parity
- Logit parity
- NLL parity
- Fixed-prompt greedy token parity
- Normal runtime imports no PyTorch or Transformers

Phase 6 — Introduce the language CL runner

Create:

```
src/apm/continual/language_tasks.py
src/apm/continual/language_run.py
src/apm/continual/language_metrics.py
```

Implement the known-boundary lifecycle:

```
training task arrives
→ exhaustive parent probe
→ freeze parent path
→ train one new LoRA edge
→ commit immutable child node
→ derive node content key
→ evaluate all learned tasks without task IDs
```

Do not modify `ModelBackend` to accept tokens.

Gate: End-to-end two-task TinyShakespeare permutation run with task-free mixed evaluation.

Phase 7 — Exhaustive prefix-energy router

Create:

```
src/apm/memory/prefix_energy.py
```

Tests:

- Constructed low-NLL adapter wins.
- Scores are normalized by valid-token count.
- Padding does not alter scores.
- Prefix-only routing cannot see suffix targets.
- Router function signature contains no task identifiers.
- Candidate masking works at every graph stage.

Gate: Exhaustive routing report for TinyShakespeare curriculum.

Phase 8 — Hopfield router

Create:

```
src/apm/memory/content_addressing.py
```

Tests:

- Normalized exact key retrieves itself.
- Batched retrieval
- Invalid-node masking
- Temperature behavior
- Top-k behavior
- Multiple examples route independently
- Base content embeddings are unaffected by graph adapters

Gate: Hopfield and exhaustive results appear in the same routing report.

Phase 9 — EBT-inspired refinement

Create:

```
src/apm/memory/address_refinement.py
```

Tests:

- One-hot address equals hard node execution.
- Invalid candidates retain zero probability.

- One optimization step has a finite gradient.
- Prefix NLL decreases on a constructed example.
- Base and edge parameters remain unchanged.
- Per-example address logits optimize independently.
- Hopfield initialization is accepted without changing the objective.
- Soft and hard results are both reported.

Gate: Uniform-start and Hopfield-start EBT results on TinyShakespeare.

Phase 10 — Curricula, baselines, and reports

Create:

```
src/apm/data/text/curricula.py
scripts/run_tinyshakespeare_cl.py
scripts/run_tinystories_cl.py
notebooks/tinystories_vamp.ipynb
```

Reuse the current JSON, JSONL, chart, heatmap, and HTML artifact infrastructure where it is domain-independent. The repository already has stable JSON/JSONL and chart writers suitable for this extraction.

Standard output:

```
results/language_cl/
    <dataset>/
        <curriculum>/
            <run_id>/
                manifest.json
                stage_metrics.jsonl
                stored_competence.jsonl
                routing_metrics.jsonl
                transfer_metrics.jsonl
                memory_metrics.jsonl
                addressing_cost.jsonl
                address_confusion.svg
                competence_curves.svg
                routing_curves.svg
                memory_curves.svg
                report.html
                samples.md
```

Gate: One reproducible TinyShakespeare baseline report and one bounded TinyStories baseline report.

---

12. Required end-to-end tests

Codex should not declare the milestone complete until these exist:

```
test_zero_effect_lora_equals_base
test_lora_path_sums_edge_contributions
test_sibling_paths_are_independent
test_only_new_edge_has_nonzero_gradients
test_base_hash_stable_after_task_training
test_committed_node_logits_stable_after_later_commits
test_path_incidence_matches_graph_paths
test_padding_does_not_change_hard_node_result
test_exhaustive_router_selects_known_best_node
test_hopfield_retrieves_known_content_key
test_ebt_one_hot_equals_discrete_node
test_ebt_refinement_reduces_constructed_prefix_nll
test_router_api_has_no_task_identity
test_address_prefix_and_evaluation_suffix_are_disjoint
test_two_task_tinyshakespeare_cl_smoke
test_checkpoint_roundtrip_preserves_logits
```

```
test_tinystories_hf_logits_parity
test_tinystories_hf_greedy_generation_parity
```

Default tests should run on CPU with tiny configurations. GPU/checkpoint/data-heavy tests should be explicitly marked integration or benchmark tests.

---

13. Non-goals and guardrails

Codex should be explicitly instructed not to:

- Build an arbitrary graph-rewrite IR.
- Support variable LoRA ranks within one graph.
- Compress adapter paths yet.
- Add LoRA to embeddings or layer normalization yet.
- Put frozen base weights into graph nodes.
- Path-sum LoRA A/B coordinates.
- Add token branches to the canvas-oriented `ModelBackend`.
- Convert existing MNIST task records into image-or-text unions.
- Add NNX.
- Make Equinox the canonical model or checkpoint representation.
- Implement a KV cache before checkpoint parity.
- Let task IDs reach any routing function.
- Evaluate routing and competence on the same token span.
- Mix JAX compilation time into steady-state latency.
- Add many command-line switches for research choices that belong in immutable experiment configurations.
- Download TinyStories or its checkpoint during ordinary unit tests.
- Preserve compatibility aliases during the generic graph refactor; the repository explicitly treats this as greenfield code.

Codex operating instruction

The implementation sequence should begin with the following directive:

> Work through `docs/LM_VAMP_EXECUTION_PLAN.md` one phase at a time. Before each phase, inspect the current graph, tests, `AGENTS.md`, `STYLE.md`, `DESIGN.md`, and `PLAN.md`. Preserve functional semantics: model parameters, optimizer state, graph state, address state, and random keys are explicit immutable values. Do not start the next phase until the current acceptance tests pass. After every phase, update `PLAN.md`; record durable architectural decisions in `DESIGN.md`; run the complete existing test suite; and report exactly which acceptance gates passed, which remain, and any measured compile/runtime behavior.

This sequence produces a validated JAX GPT-Neo implementation first, then establishes LoRA path semantics on a cheap character model, then crosses the PyTorch checkpoint boundary exactly once, and only afterward builds the three addressing methods and the full continual-learning benchmark.